METRISCAN AI

SIH Project Blueprint — From Problem Statement to Working Prototype

Problem Statement ID: 26034

Organization: Ministry of Consumer Affairs, Food & Public Distribution

Department: Department of Consumer Affairs (DoCA)

Theme: Software

Product: AI-assisted packaged commodity compliance verification platform.

1. Problem in Simple Words

Legal Metrology enforcement officers need to inspect packaged commodities for mandatory label declarations. Manual inspection is repetitive and time-consuming.

MetriScan AI assists officers by turning package photographs into structured declarations, rule evaluations, evidence, inspection history and reports.

2. Core Product Flow

```
PRODUCT PACKAGE → IMAGE SCAN → IMAGE PROCESSING → OCR → FIELD EXTRACTION → RULE ENGINE →
PASS / FAIL / REVIEW → EVIDENCE → REPORT → DASHBOARD
```

3. Primary Users

Enforcement Officer: creates inspections, uploads images, reviews findings and generates reports.

Reviewer / Senior Officer: reviews uncertain or flagged cases and finalizes findings.

Administrator: manages users, rules, rule versions, categories, analytics and audit activity.

4. MVP Scope

Secure authentication and role-based access.

Inspection creation with product details, date/location and multiple images.

Image quality assessment, preprocessing, OCR, field extraction and confidence scoring.

Mandatory declaration detection and validation.

Rule engine with PASS, FAIL, REVIEW and NOT APPLICABLE.

Evidence view with image regions, OCR text and rule explanation.

PDF and structured report generation.

Searchable inspection/product history.

Enforcement dashboard and analytics.

5. Out of Scope for MVP

Complete FSSAI, BIS or every other Indian regulatory framework.

Physical weighing verification of actual contents.

Automatic legal enforcement action.

Every Indian language from day one.

Complete e-commerce scraping.

Fully autonomous legal decisions.

Complex microservice/Kubernetes infrastructure.

6. Recommended Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite + Tailwind CSS	Officer UI, dashboard, inspection and results
Backend	Python + FastAPI	REST APIs, orchestration, authentication

AI / CV	OpenCV + PaddleOCR	Image processing and OCR
Extraction	Python regex + heuristics; optional NLP/LLM assistance	Structured field extraction
Database	PostgreSQL	Users, inspections, rules, violations and history
ORM/Migrations	SQLAlchemy + Alembic	Database access and migrations
Security	JWT + Argon2/bcrypt	Authentication and RBAC
Reports	Python PDF generation	Inspection/compliance reports
DevOps	GitHub + Docker	Version control and reproducibility

7. System Architecture

```
OFFICER / ADMIN / REVIEWER ↓ REACT FRONTEND ↓ FASTAPI BACKEND ■■■ Authentication & RBAC ■■■ Inspection Module ■■■ Image Module ■■■ AI / OCR Module ■■■ Field Extraction Module ■■■ Compliance / Rule Engine ■■■ Report Module ■■■ Dashboard Module ↓ POSTGRESQL + FILE / OBJECT STORAGE
```

For SIH, start with a modular monolith. It is easier to build, test and deploy than a large microservice architecture.

8. AI / OCR Pipeline

```
IMAGE → QUALITY CHECK → PREPROCESSING → OCR → TEXT + BOUNDING BOXES + CONFIDENCE → FIELD EXTRACTION → NORMALIZATION → RULE ENGINE
```

- Detect blur, poor lighting and unreadable captures.
- Apply resize, orientation correction, perspective correction, denoising, contrast enhancement and sharpening.
- Return OCR text, confidence and bounding boxes.
- Extract MRP, net quantity, dates, manufacturer, address, consumer care and other applicable fields.
- Normalize equivalent representations such as 500g and 500 g.
- Use confidence thresholds to trigger human review instead of guessing.

9. Compliance Rule Engine

AI answers: “What does the label say?” Rule Engine answers: “Does it comply with the applicable requirement?”

```
INPUT → APPLICABILITY ENGINE → FIELD / FORMAT / READABILITY VALIDATORS → RESULT AGGREGATOR → PASS / FAIL / REVIEW / NOT APPLICABLE
```

Rule source: actual legal conditions must be populated from current official Department of Consumer Affairs material. Do not invent legal rules or hard-code an outdated 2011 snapshot.

10. Core Database Model

```
USER → INSPECTION → PRODUCT / IMAGES → OCR_RESULTS → EXTRACTED_FIELDS → RULE_RESULTS → VIOLATIONS → REPORT RULE → RULE_VERSION → RULE_RESULT
```

- users
- products
- inspections
- images
- ocr_results
- extracted_fields
- rules
- rule_versions
- rule_results
- violations
- reports
- audit_logs

11. API Plan

```
POST /api/auth/login GET /api/auth/me POST /api/inspections GET /api/inspections GET
/api/inspections/{id} POST /api/inspections/{id}/images POST /api/inspections/{id}/analyze GET
/api/inspections/{id}/results GET /api/inspections/{id}/violations POST
/api/inspections/{id}/report GET /api/rules POST /api/rules PUT /api/rules/{id} GET
/api/dashboard/summary GET /api/dashboard/trends
```

12. Backend Project Structure

```
backend/ ■■■ app/ ■ ■■■ main.py ■ ■■■ api/ # auth, inspections, rules, reports, dashboard ■ ■■■ models/ # database models ■ ■■■ schemas/ # request/response schemas ■ ■■■ services/ # OCR, extraction, compliance, reports ■ ■■■ rules/ # rule loading/evaluation ■ ■■■ core/ # config, security, database ■■■ tests/
```

13. AI Project Structure

```
ai/ ■■■ preprocessing/ ■ ■■■ quality.py ■ ■■■ resize.py ■ ■■■ deskew.py ■ ■■■ enhance.py ■■■ ocr/ ■ ■■■ paddleocr_service.py ■■■ extraction/ ■ ■■■ mrp.py ■ ■■■ quantity.py ■ ■■■ dates.py ■ ■■■ manufacturer.py ■ ■■■ consumer_care.py ■■■ confidence/ ■■■ scoring.py
```

14. Inspection Result Design

```
PRODUCT: ABC Biscuits NET QUANTITY: 500 g — 98% confidence — PASS MRP: ■120 — 97% confidence — PASS MANUFACTURER: ABC Foods Pvt Ltd — 96% confidence — PASS CONSUMER CARE: Not detected — FAIL PACKED DATE: 06/2026 — 58% confidence — REVIEW FINAL STATUS: NON-COMPLIANT
```

15. Evidence & Explainability

- Show original image and relevant detected region.
- Show OCR text, extracted field and confidence.
- Show applicable rule identifier/version.
- Explain the reason for PASS/FAIL/REVIEW.
- Allow human correction for low-confidence or incorrect OCR.
- Store corrections and an audit trail.

16. Dataset Strategy

Start with packaged FMCG/food products. Collect compliant examples and create controlled synthetic test cases for missing or unreadable declarations. Every test case should have manually verified ground truth.

```
dataset/ ■■■ compliant/ ■■■ test_cases/ ■■■ missing_mrp/ ■■■ missing_quantity/ ■■■ missing_manufacturer/ ■■■ unreadable_text/ ■■■ missing_consumer_care/
```

17. Testing Strategy

- OCR and field extraction accuracy.
- Rule PASS, FAIL, REVIEW and NOT APPLICABLE cases.
- Rotation, blur, low-light and perspective-distortion tests.
- API authentication, permissions and inspection lifecycle tests.
- End-to-end upload → OCR → extraction → rules → result → PDF tests.
- Maintain an initial set of roughly 30–50 manually verified golden test cases.

18. Development Roadmap

Phase	Focus	Indicative Duration
0	Requirements, rule inventory, architecture, wireframes	1–2 days
1	Official rule research and rule data model	2–4 days

2	OCR + image preprocessing prototype	3–5 days
3	Field extraction + normalization	3–5 days
4	Rule engine	3–5 days
5	PostgreSQL + FastAPI backend	4–6 days
6	React frontend	5–7 days
7	Integration + evidence + PDF	3–5 days
8	Testing, deployment and SIH polish	3–5 days

19. Team Division

Member	Primary Ownership
1	React frontend, screens, dashboard, result visualization
2	FastAPI backend, authentication, APIs and reports
3	OpenCV, OCR, image preprocessing, extraction and confidence
4	Legal rule research, rule repository, applicability and rule engine
5	PostgreSQL, ER design, migrations, history and analytics
6	Integration, GitHub, Docker, testing, deployment and demo

20. Critical Design Principles

- **AI extracts; rules decide.** Do not use an LLM as the sole legal decision maker.
- **Human-in-the-loop.** Low-confidence cases must be reviewable.
- **Rule versioning.** Legal requirements change, so rules need versions and effective dates.
- **Evidence-first.** Every violation should be traceable to an image/text region and rule.
- **Scope discipline.** MVP should focus on Legal Metrology packaged-commodity declarations.
- **Modular monolith first.** Keep architecture simple enough for an SIH team to finish.
- **Auditability.** Store who created, reviewed, changed and finalized an inspection.

21. SIH Demo Flow

1. Officer logs in. 2. Creates a new inspection. 3. Uploads front/back package images. 4. System checks image quality. 5. OCR extracts text and bounding boxes. 6. Fields appear with confidence scores. 7. Rule engine evaluates applicable requirements. 8. Result shows PASS / FAIL / REVIEW. 9. Judge opens a violation and sees highlighted evidence plus the rule explanation. 10. Officer generates a professional PDF report. 11. Dashboard updates statistics. 12. Inspection is searchable in history.

22. First Technical Milestone

```
Before building the dashboard, make a Python prototype work end-to-end: product.jpg → preprocessing → OCR → structured fields → rule engine → PASS/FAIL/REVIEW Once this “brain” works reliably, build the database, backend and frontend around it.
```

23. Final Product Statement

MetriScan AI is an explainable, rule-versioned, AI-assisted inspection platform that extracts declarations from packaged commodity labels, evaluates them against applicable Legal Metrology rules, preserves visual evidence and inspection history, supports human review, and generates enforcement-ready compliance reports.

Next: We will move to the detailed PRD and system architecture: user stories, use cases, complete functional/non-functional requirements, data-flow diagrams, ER diagram, database schema, API contracts, frontend page map, AI pipeline and rule-engine design.